

XProc 3.0 -Connecting step using ports

January 23, 2020

Erik Siegel (erik-siegel@nortek.no)

Erik Siegel continues his series of articles on XProc 3.0, this time focusing on connecting steps.

Table of Contents

Introduction

What are ports?

Declaring ports

Input defaults

Output connections

Connecting ports binding ports

Connecting from the outermost step

Implicit connections

Explicit connections

Wrap up

Introduction

XProc is an XML-based language for processing documents in pipelines and other transformations. This article is part of a series of articles on XProc 3.0, which can be found at <http://www.xml-cs.org/xproc/2010-01-23-01/>.

This article is part of a series of articles on XProc 3.0, which can be found at <http://www.xml-cs.org/xproc/2010-01-23-01/>. The series is intended to provide a comprehensive overview of the XProc 3.0 specification, and to show how to use the various features of the language. The first article in the series, "What are ports?", is a good starting point for anyone new to XProc 3.0.

What are ports?

As explained in the introduction, XProc 3.0 is a language for processing documents in pipelines and other transformations. The basic building blocks of XProc 3.0 are steps, which are used to perform various operations on documents. Steps are connected to each other using ports, which are the points of entry and exit for data. The data is represented as XML documents, and the steps are used to transform these documents in various ways. The first step in a pipeline is the input step, which takes the input documents and passes them to the next step. The last step in a pipeline is the output step, which takes the output of the previous step and passes it to the next step.

Ports are the points of entry and exit for data in a pipeline. They are used to connect steps to each other, and to pass data between them. There are two types of ports: input ports and output ports. Input ports are used to pass data into a step, and output ports are used to pass data out of a step. The data is represented as XML documents, and the steps are used to transform these documents in various ways.

For instance, the `<xsl:transform>` step has two input ports: the document it also has two output ports: the transformed document and one additional document(s) with the result - document



(/media/kmads/ksatmp/wei-dptchr-1280.png)

Figure 1: The `p : x s` step with its in- and output ports

Ports are defined with the `<p:output>` element as follows (simplified declarations of `<xf:transform>` step):

Example 1. Simplified port declarations for the `<p : x s` step

```

p input port    source    primary    true    sequence    true
p input port    stylesheet
p output port    result    primary    true
p output port    secondary    sequence    true
  
```

Declaring ports

Now `<p:xslt>` is defined, the ports are declared as follows (http://spec.xproc/master/head/tutorials/step.html). You can now write your own steps, which can be added to the `<xproc>` namespace.

Declarations of inputs and outputs of the `<p:xslt>` step are in the `<p:declare-ports>` block, but the steps constituting the `<p:declare-ports>` block are in the `<p:body>` block (see the `<xproc>` namespace).

Example 2. An XProc step divided into a prolog and a body

```
p declare-step xmlns p http . 3.org ns xproc
```

```
-- tep s prolog --  
p input port source  
p output port result
```

```
-- tep s bod --  
p xslt  
p ith-input port st lesheet href do-something.xsl  
p xslt
```

```
p declare-step
```

The following attributes are used to declare them:

port

The mandatory attribute is name. for the port

By convention the input and output ports are named result

primary

Setting this boolean attribute to true at primary is a primary role in connecting to the source. See the "Implicit connection" section.

There can only be one primary port in a step. The primary port is the only port that can be connected to. The primary port is the only port that can be connected to. The primary port is the only port that can be connected to.

sequence

This boolean attribute indicates whether the sequence of steps is ordered. If true, the steps are ordered. If false, the steps are unordered. The sequence of steps is ordered if the attribute is set to true.

content-types

This attribute specifies the content types (MIME types) of the input and output documents. The attribute is optional. If present, it must be a list of MIME types separated by spaces. The attribute is optional. If present, it must be a list of MIME types separated by spaces.

The exact rules for the content-types attribute are defined in the XML Schema specification. The attribute is optional. If present, it must be a list of MIME types separated by spaces. The attribute is optional. If present, it must be a list of MIME types separated by spaces.

serialization

This attribute is only for the `xml:serialize` attribute. When the `xml:serialize` attribute is present on an element, it indicates that the element's content should be serialized. The value of the `xml:serialize` attribute can be `text`, `xml`, or `base64`. The default value is `text`.

The value of the `xml:serialize` attribute can be `text`, `xml`, or `base64`. The default value is `text`. The `xml:serialize` attribute is only for the `xml:serialize` attribute.

Input port defaults

It is possible to define a default connection for an input port. This is useful when you have a large number of input ports and you want to define a default connection for all of them. The default connection is defined by the `default-st` attribute.

Example 3. Defining an input port with a default connection

```
p input name special-st lesheet
p document href default-st lesheet.xml
p input
```

If you don't connect anything to the `special-st` port, the default connection will be ignored. The default connection is defined by the `default-st` attribute.

The rules and syntax for defining a default connection are the same as for defining a connection. The only difference is that the `default-st` attribute is used instead of the `st` attribute.

Output port connections

An output port can be connected to one or more other output ports. This is useful when you have a large number of output ports and you want to define a default connection for all of them. The default connection is defined by the `default-st` attribute.

For the purpose of this document, we will usually use the `default-st` attribute to define a default connection. The `default-st` attribute is defined by the `default-st` attribute.

Example 4. Defining an input port with a default connection

```
p declare-step xmlns="http://www.w3.org/1999/xproc"
```

```
p input port source
p output port result
p output port extra pipe result transform-step
```

```
p xslt name transform-step
p with-input port stylesheet href do-something.xsl
p xslt
```

```
p declare-step
```

The extra output ports always exist, even if the pipeline goes on after this.

Connecting or binding ports

To make pipelines work, you need to connect the input ports of the first step to the output ports of the previous step. This is done by using the `with-input` attribute of the `step` element. For example, the following pipeline connects the `source` input port of the `transform-step` to the `result` output port of the `input` step.

Connecting ports for the outermost step

When you write the outermost step of a pipeline, you need to bind its input ports to something (usually a file or a string). This is done by using the `with-input` attribute of the `step` element. For example, the following pipeline binds the `source` input port of the `transform-step` to the `result` output port of the `input` step.

Unfortunately, the XProc specification does not provide any guidance on how to connect the input ports of the outermost step to something. This is a common problem for XProc developers.

Implicit connections

The XProc specification (http://www.w3.org/TR/xproc-2010-01/) already explained implicit binding. Here is an example:

```
<input port="source" type="text" value="Hello, world!" />
<transform-step name="uppercase" />
```

Not only does implicit binding work, but it is also the only way to connect the input ports of the outermost step to something. This is a common problem for XProc developers.

some-step

```
p    ith-input port    source
p    pipe step    step-to-connect-to    port    secondar
p    ith-input
some-step
```

Exactly the same over each widget hierarchy

Example 7. Connect to an output port using the `pipe` attribute

```
some-step
p    ith-input port    source    pipe    secondar    step-to-connect-to
some-step
```

Something reachable by URI

For this `<p:document href="some-file.xml">` element the attribute:

Example 8. Connect to a URI port using `<p:document>`

```
some-step
p    ith-input port    source
p    document href    some-file.xml
p    ith-input
some-step
```

Exactly the same over each widget hierarchy

Example 9. Connect to a URI using the `href` attribute

```
some-step
p    ith-input port    source    href    some-input.xml
some-step
```

In the example above, `some-file.xml` will use a global namespace location. An absolute file name `/` must be provided.

Something stated inline

If the document type is a template, it can state it as `<p:inline>` attribute. A template is a `<p:template>` element between `{ ... }` expanded:

Example 10. State an input document in the XProc code itself

```

p      ariable name      debug      select      true
some-step
p      ith-input port      source
p      inline
input-document timestamp      current-date      ime
p      Debug setting      debug      p
input-document
p      inline
p      ith-input
some-step

```

If the document is seen in XML headers, only receive a block of the document.
 <p:inlinel element.

Nothing

Sometimes at the end of a document, you can have an empty element case.

Example 11. Connect an input port to nothing using <p:empty>

```

some-step
p      ith-input port      source
p      empty
p      ith-input
some-step

```

Wrap up

Understand the design of the document, the main aim is to understand the structure of the document.

- You can use the [XPath](#) to find the elements in the document.
- You can use the [XPath](#) to find the elements in the document, so the document is not empty.

In other words, if you get the document, you can use the XPath to find the elements in the document.

Article contents © 2020 Erik Siegel

Related links

- [An introduction to XPath 3.1 \(Part 3\) \(Articles / 2019/01/05/chr-310p/r\)](#)

© Textuality Services, Inc. except for the trademarks and service marks appearing on the cover of the book, all other trademarks and service marks appearing on the cover of the book are the property of their respective owners.